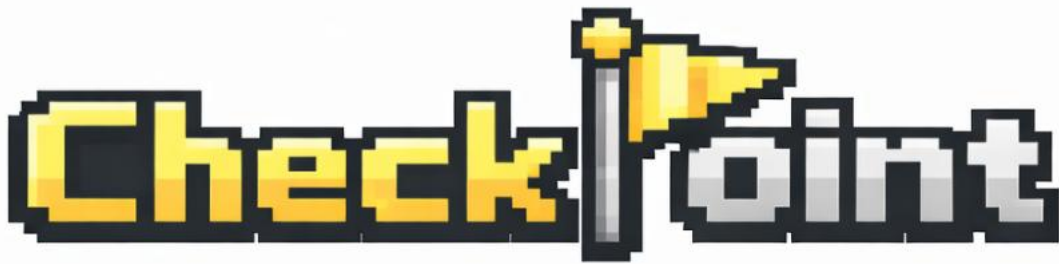


CheckPoint



Informe de Implementación de la GUI

Tienda de venta friki e intercambio de productos.

Graphic User Interface

Antonino Albarrán Peñas · Daniel González Ureta · Lucas Blanco Rodríguez

PADSOF - Curso 2025/2026

Índice

Introducción.....	3
Decisiones de implementación	4
Empleo de controladores.....	7
Información adicional	9

Introducción

En este apartado vamos a explicar las principales decisiones que hemos tomado al implementar la interfaz gráfica de la aplicación. La idea no es solo comentar qué pantallas tiene cada usuario, sino también justificar un poco cómo las hemos organizado, por qué hemos reutilizado ciertas clases y cómo hemos separado la parte visual de la lógica mediante controladores.

Para ello, hemos dividido la explicación según los distintos tipos de usuario de la tienda: cliente registrado y no registrado, empleado y gestor. Cada uno tiene unas funcionalidades diferentes, por lo que también su interfaz cambia según lo que puede hacer dentro del sistema. Aun así, hemos intentado que todas las pantallas sigan una estructura parecida, usando paneles, tablas, botones, filtros, scrolls y formularios con un estilo común.

También comentaremos el uso de clases base, herencia y métodos reutilizables, ya que han sido importantes para no repetir código y para mantener una estética uniforme en toda la aplicación. Además, explicaremos brevemente el papel de los controladores, que nos han permitido separar mejor la interfaz de las operaciones reales que se hacen sobre la tienda.

Decisiones de implementación - Cliente (Reg y NoReg)

En la parte del cliente separamos el cliente registrado del no registrado, porque no pueden hacer las mismas cosas. El invitado tiene una interfaz más simple: puede ver el catálogo, consultar productos de segunda mano e ir al login o al registro. En cambio, el cliente registrado tiene muchas más opciones, como carrito, pedidos, cartera, intercambios, notificaciones, perfil y descuentos. Aun así, reutilizamos algunas pantallas, como catálogo y segunda mano, para no repetir código y mantener una estructura parecida en ambos casos.

El cliente registrado usa un CardLayout, igual que empleado y gestor. Esto nos permite tener varias secciones dentro del mismo panel y cambiar entre ellas con los botones de arriba. Cuando el cliente pulsa una pestaña, se muestra esa parte y se actualizan sus datos, por ejemplo el carrito, los pedidos o las notificaciones. Así no hace falta abrir ventanas nuevas y todo queda dentro de la misma pantalla principal.

En el catálogo mostramos los productos como tarjetas, con imagen, nombre, precio, stock y puntuación.

También añadimos filtros y ordenación para buscar mejor, sobre todo pensando en que pueda haber muchos productos. Si el usuario está registrado, puede ver recomendaciones según su historial; si es invitado, se le muestran productos destacados o mejor valorados. Al abrir un producto se ve su detalle completo, y solo si el cliente está registrado aparece la opción de añadirlo al carrito.

El carrito está hecho con los productos a un lado y un resumen al otro, donde se ve subtotal, descuento, total y tiempo restante. Desde ahí se pueden cambiar cantidades, eliminar productos y reservar el pedido. En pedidos, el cliente puede ver su historial, pagar pedidos pendientes, pedir recogida y escribir reseñas de productos ya entregados. También controlamos los tiempos para que no se pueda pagar o reservar algo caducado, porque eso podría dejar datos antiguos o stock mal reservado.

La parte de segunda mano la hicimos separada del catálogo normal porque funciona distinto. Aquí los productos tienen propietario, tasación, estado y pueden estar bloqueados o visibles. El cliente registrado puede subir productos, pedir tasación y hacer ofertas de intercambio. El invitado solo puede consultar lo visible, pero no puede hacer acciones que necesiten cuenta, como proponer ofertas o gestionar una cartera propia.

Decisiones de implementación - Empleado

La parte del empleado la hemos organizado como un panel principal desde el que se puede ir cambiando entre distintas secciones, como inventario, editar producto, categorías, packs, pedidos, entregas, tasaciones, intercambios y notificaciones. Para esto usamos un CardLayout, que básicamente nos permite tener varias pantallas cargadas dentro del mismo panel y enseñar solo una cada vez. Arriba hay una barra de navegación con botones, y cada botón cambia a una sección distinta. Además, esas secciones no aparecen siempre, sino que se muestran según los permisos que tenga el empleado. Por ejemplo, si un empleado no tiene permiso para gestionar packs o pedidos, directamente no le sale esa opción. Esto nos sirvió bastante para manejar los permisos de forma más fácil dentro de la interfaz.

Para no repetir el mismo código en todas las pantallas, fuimos sacando lo común a clases base. La más general es PanelBaseInterfaz, donde dejamos métodos que se usan en casi toda la GUI, como crear campos, botones, combos, etiquetas, bloques, scrolls, mensajes de error o mensajes de confirmación. También se encarga de mantener un estilo parecido en todas las pantallas. Luego, para la parte concreta del empleado, usamos SeccionEmpleadoBase, que guarda la ventana principal y el empleado que ha iniciado sesión. Así cada sección no tiene que estar recibiendo o buscando esos datos por separado.

Como muchas pantallas del empleado trabajan con productos, hicimos otra clase base más concreta, SeccionProductosVentaEmpleadoBase. Ahí dejamos una tabla común de productos de venta, con columnas como ID, nombre, tipo, categorías, precio, stock y puntuación. Esa tabla ya trae filtros por texto, por tipo de producto y por categoría, además de un desplegable para ordenar por nombre, precio, stock o puntuación. Esto lo reutilizamos en secciones como inventario, categorías, editar producto y packs, así que no tuvimos que crear una tabla diferente desde cero para cada pantalla.

La interfaz del empleado usa principalmente una estructura de tabla arriba y bloque de acciones abajo. En inventario, por ejemplo, primero se ve la tabla con los productos actuales y después hay bloques para sumar o restar stock, cargar productos desde fichero y crear productos nuevos. Para crear productos no pusimos todos los campos mezclados, porque quedaría bastante lioso. Hay una parte de datos comunes, como nombre, descripción, imagen, precio, stock y categorías, y luego una parte que cambia según el tipo de producto. Para eso usamos un CardLayout: si se elige cómic, aparecen los campos de cómic; si se elige juego, aparecen los de juego; y si se elige figura, aparecen los de figura. Además, para las imágenes usamos botones para seleccionar o ver la imagen, y una utilidad aparte que normaliza la ruta y busca la imagen en src/fotos.

Decisiones de implementación - Gestor

La parte del gestor también la organizamos con un panel principal y un CardLayout, igual que en los demás usuarios. Desde la barra de arriba podemos cambiar entre las secciones de empleados, categorías, productos y descuentos, estadísticas, configuración y perfil. En este caso no hace falta ocultar secciones por permisos, porque el gestor tiene acceso a toda la información de la tienda.

Para reutilizar código usamos AbstractPanelGestor, que hereda de PanelBaseInterfaz. Así aprovechamos los mismos métodos para crear botones, campos, ..., y además añadimos algunos elementos solo para el gestor, como campos con columnas o campos de contraseña. Esto hace que los subpaneles reutilicen muchas líneas de código y siempre tengan un estilo parecido.

En la sección de empleados, el gestor puede dar de alta empleados nuevos, seleccionar sus permisos con boxes y luego consultar la lista de empleados registrados. En cada fila vemos su info y desde ahí se pueden añadir o quitar permisos o despedir un empleado. También añadimos búsqueda y filtros por permisos para que sea más cómodo cuando haya muchos más empleados.

En categorías y productos/descuentos seguimos una estructura parecida a la que tenemos en empleado, ya que primero se muestran los productos de venta y luego aparecen bloques para hacer las acciones. En categorías se pueden crear, eliminar y asignar o quitar a productos usando su ID. En productos y descuentos reutilizamos la tabla común de productos, permitimos cambiar precios y crear descuentos. Para los descuentos usamos un CardLayout, de forma que según el tipo de descuento aparezcan unos campos u otros.

Por último, el gestor tiene partes más generales de configurar e información de la tienda. En estadísticas se pueden ver los ingresos, rankings y consultas por fechas; en configuración se modifican tiempos de caducidad, precio de tasación de productos de segunda mano y pesos del recomendador; y en perfil podemos cambiar el nickname y contraseña.

Empleo de controladores - Cliente (Reg y NoReg)

En cliente también usamos controladores para no meter toda la lógica en los botones. La pantalla enseña los campos, tarjetas y mensajes, y el controlador decide qué hacer cuando el usuario pulsa algo. Por ejemplo, `ControladorPanelCliente` cambia entre secciones del cliente registrado, y `ControladorPanelInvitado` hace lo mismo pero con las opciones más básicas del invitado. Con esto la vista queda más limpia y no se mezcla tanto el diseño con las operaciones.

El `ControladorCatalogo` se encarga de buscar productos, aplicar filtros, ordenar, abrir detalles y añadir al carrito cuando hay cliente registrado. Como lo usamos también para invitados, simplemente se bloquean o no aparecen las acciones que necesitan cuenta. Así reutilizamos la misma idea de catálogo para los dos tipos de usuario. Esto nos evitó tener dos catálogos casi iguales programados por separado.

Luego hay controladores para partes más concretas. `ControladorCarrito` gestiona cantidades, eliminación de productos y reserva del carrito, comprobando antes si ha caducado. `ControladorPedidos` gestiona pagos, recogidas y reseñas, revisando también que el pedido esté en el estado correcto. En segunda mano hay controladores para ver productos, crear ofertas, aceptar o rechazar intercambios y pagar tasaciones. Cada uno se ocupa de una parte concreta para que no quede todo acumulado en una sola clase.

Empleo de controladores - Empleado

En el empleado, los controladores los usamos para que cada parte de la interfaz tenga claro qué tiene que hacer. Por ejemplo, el de stock se ocupa de sumar o quitar unidades y de cargar productos desde fichero; el de categorías sirve para añadir o quitar una categoría a un producto; el de packs lleva todo lo de crear packs, cambiar unidades, modificar precios o eliminarlos; y el de modificar producto permite cargar un producto por su ID y cambiar sus datos. Así no metemos toda esa lógica directamente en los botones de la pantalla.

También hay controladores para las partes más de trabajo diario del empleado, como preparar pedidos, entregar pedidos con el código de recogida, tasar productos de segunda mano o confirmar intercambios. En estas secciones el controlador comprueba cosas antes de hacer la acción, como si el ID existe, si el pedido está en el estado correcto o si el empleado tiene permiso. De esta forma, cada pantalla queda más limpia y cada controlador se encarga de una tarea.

Empleo de controladores - Gestor

En los controladores del gestor hay algunas cosas un poco más concretas porque el gestor toca muchas partes distintas de la tienda. Por ejemplo, el panel principal del gestor usa un `CardLayout`, pero el cambio entre secciones no lo hace directamente cada botón, sino `ControladorPanelGestor`. Cuando pulsamos una pestaña, este controlador le dice al panel qué sección tiene que enseñar y también cuál tiene que quedar marcada como activa. Así la navegación queda separada del diseño de la pantalla y no se mezcla todo en el mismo sitio.

Luego, cada zona del gestor tiene su propio controlador. En categorías, por ejemplo, se usan acciones para crear, añadir, quitar o eliminar categorías, y además hay un método para comprobar si un producto ya tiene una categoría antes de tocarlo, para evitar errores raros al usar la interfaz. También reutilizamos cosas del controlador de productos del empleado para no volver a programar métodos como sacar el tipo de producto, sus categorías, el precio o la puntuación. En productos y descuentos pasa parecido: el controlador mira si se quiere cambiar un precio, crear un descuento o eliminarlo, y llama al método del gestor que corresponde. Y cuando algo cambia de verdad en la tienda, normalmente se llama a `GuardadoTienda.guardar(tienda)`, para que los cambios no se pierdan al cerrar el programa.

Información adicional

Hemos intentado que la interfaz sea lo más extensible posible desde dos puntos de vista. Por un lado, hemos añadido bastantes scrolls, filtros y opciones de ordenación para que la aplicación pueda manejar una cantidad grande de productos, pedidos, descuentos o notificaciones sin que la pantalla se vuelva incómoda.

Por otro lado, la forma en la que está programada la GUI permite que, si en un futuro se quiere añadir una nueva funcionalidad, sea más sencillo hacerlo. Esto se debe a que contamos con varias clases base de las que podemos heredar y con métodos estáticos para crear elementos de la interfaz que ya siguen la misma estética y funcionamiento que el resto de la tienda.

Otro aspecto que hemos cuidado bastante es que la interfaz no solo funcione, sino que también sea visualmente bonita e intuitiva. En algunas partes nos hemos inspirado en tiendas online tipo Amazon, sobre todo en la forma de mostrar productos y sus detalles. Aunque muchas secciones están organizadas en bloques sencillos, hemos intentado que dentro de esos bloques la información esté bien puesta y sea cómoda para el usuario. Para conseguirlo, hemos hecho métodos que devuelven componentes de la GUI ya formateados, como botones, campos, combos, paneles o scrolls, usando siempre los colores, bordes, fondos y fuentes principales de la tienda. Estos colores los hemos centralizado como constantes en la ventana principal, para mantener una estética común en toda la aplicación.

También creemos que hemos tenido en cuenta muchos detalles pequeños que mejoran bastante el uso de la aplicación. Por ejemplo, se actualizan los tiempos y estados al cambiar entre secciones como carrito o pedidos, las notificaciones se muestran primero con las más recientes, y usamos un método de escalado para adaptar mejor los tamaños a los DPI del ordenador. También hemos controlado casos concretos, como que los descuentos de regalo no se muestren al cliente si ya no quedan unidades del producto regalado, para evitar confusiones.

Además, hemos añadido varias funciones prácticas para cada tipo de usuario. El empleado puede abrir imágenes de productos, cargar imágenes desde archivos del ordenador y gestionar productos, stock, pedidos o tasaciones. El gestor puede consultar diferentes tipos de ingresos, filtrarlos por rangos de fechas, revisar ingresos de otros años y modificar parámetros generales de la tienda. En general, creemos que hemos dedicado bastante tiempo y cuidado a que la aplicación no solo cumpla con las funcionalidades pedidas, sino que también tenga una interfaz clara, útil y lo más completa posible.